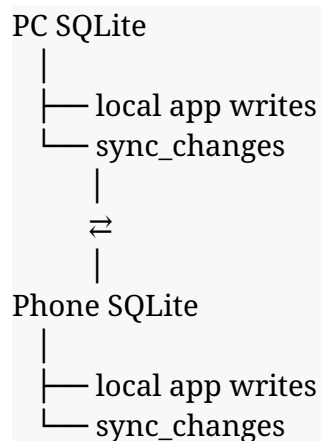


We solve it by treating **sync as a separate system from the database itself**.



SQLite can hold the local data on each device, but the real magic is a **change log + conflict detection + merge policy**.

The clean v1 design is:



The key idea is: **never sync raw database files**. Sync **record changes**.

Your audit already concluded the same thing: basic record metadata is enough for simple synchronization, but proper conflict handling also needs change IDs, base versions, tombstones, and per-peer cursors.

First rule: every device gets an identity

PC:

deviceId = PC-UUID

Phone:

deviceId = PHONE-UUID

Every persistent record has:

id
createdAt
updatedAt
deletedAt

version

```
originDeviceId  
lastModifiedByDeviceId
```

Example trade:

```
id = trade-abc  
version = 4  
lastModifiedByDeviceId = PC
```

Second rule: every write creates a change record

Suppose you edit a trade on the PC.

The trade goes from:

```
version 4
```

to:

```
version 5
```

At the same time FXJourney writes:

```
sync_changes
```

```
---
```

```
changeId  
entityType = TRADE  
entityId = trade-abc
```

```
operation = UPDATE
```

```
baseVersion = 4  
newVersion = 5
```

```
deviceId = PC
```

```
createdAt  
payloadJson
```

That baseVersion is extremely important.

It tells the other device:

“This edit was made from version 4.”

That is how we detect conflicts.

Normal sync is easy

Suppose both devices last had:

Trade ABC
version 4

Then the phone stays offline.

PC edits it:

PC
version 4
→ version 5

Phone reconnects and still has:

version 4

The PC sends:

baseVersion = 4
newVersion = 5

Phone checks:

my version = 4
incoming base = 4

They match.

So:

safe update

Phone becomes version 5.

Done. 🕶️

The conflict case

Now imagine this:

Both start with:

```
Trade ABC  
version 4
```

PC goes offline and edits:

```
entryPrice = 4350  
version 5  
baseVersion = 4
```

Phone also edits while disconnected:

```
entryPrice = 4360  
version 5  
baseVersion = 4
```

Then reconnect.

PC sends its change.

Phone sees:

```
incoming baseVersion = 4  
my current version = 5
```

Mismatch.

That means:

Both devices independently edited the same old version.

 **Conflict detected.**

We do **not** silently overwrite either one.

Different data types need different conflict rules

This is the important part.

We should not treat everything the same.

1. Immutable data

Examples:

```
analysis_report_versions
```

These are append-only.

So:

```
Report A created on PC
```

```
Report B created on phone
```

Both survive.

No conflict needed.

The audit specifically recommends immutable Analyzer reports as append-only.

2. Simple independent fields

Example:

PC changes:

```
trade.rating
```

while phone changes:

```
trade.notes
```

Technically both edited the same trade record, but they modified different fields.

We can merge them automatically:

PC:

```
rating = 5
```

Phone:

```
notes = "Waited for confirmation"
```

Final:

```
rating = 5
```

```
notes = "Waited for confirmation"
```



This is much better than whole-record last-write-wins.

So our sync payload should preferably know **which fields changed**.

Example:

```
changedFields:  
[  
  "rating"  
]
```

instead of only sending the entire object blindly.

3. Same field edited on both devices

Now:

PC:

```
entryPrice = 4350
```

Phone:

```
entryPrice = 4360
```

Same field.

This should be a real conflict.

FXJourney stores:

```
sync_conflicts
```

```
---
```

```
id  
entityType  
entityId
```

```
fieldName
```

```
localValue
```

remoteValue

localDeviceId
remoteDeviceId

localVersion
remoteVersion

detectedAt

status

Then UI can later show:

Sync Conflict

Entry Price

Main PC:
4350

Blade A75:
4360

Keep:
[PC value] [Phone value]

This is much safer for financial data.

4. Tags and categories

These shouldn't overwrite the entire array.

Bad:

```
tags = ["London", "Breakout"]
```

syncing as one blob.

Because PC might add:

London

while phone adds:

```
A+ Setup
```

and one would wipe the other.

Instead, use relationship records:

```
trade_tags
```

Each relationship has its own identity/tombstone.

So:

```
PC:
```

```
ADD trade-abc + tag-london
```

```
Phone:
```

```
ADD trade-abc + tag-aplus
```

after sync:

```
London
```

```
A+ Setup
```

Both survive.

The audit specifically warns that junction relationships should sync independently.

5. Deletes

Never immediately hard-delete synced records.

Suppose you delete a journal entry on phone while PC is offline.

If we physically erase it:

```
journal row gone
```

the PC can't know it should also delete it.

Instead:

```
deletedAt = timestamp
```

and create:

```
operation = DELETE
```

That is a **tombstone**.

PC reconnects:

```
receive DELETE  
→ set deletedAt
```

Then both devices agree.

Much later, once we know all devices have seen the deletion, we can permanently purge the record if we want.

6. Delete vs edit conflict

Interesting case:

PC:

```
edit Trade ABC
```

Phone:

```
delete Trade ABC
```

while both offline.

When they reconnect, don't automatically choose.

That becomes:

```
DELETE_EDIT_CONFLICT
```

UI might say:

```
This trade was deleted on Blade A75  
but modified on Main PC.
```

```
[Keep deleted]
```

```
[Restore edited version]
```

We preserve the data until the user decides.

7. Habits need a special rule

Habit entries are naturally identified by:

(habitId, entryDate)

The audit explicitly recommends that uniqueness.

Example:

habitId = no-revenge-trading

entryDate = 2026-09-13

Only one logical record exists for that day.

If PC says:

COMPLETED

and phone says:

SKIPPED

same date, same habit:

conflict

Don't duplicate both rows.

8. Journal text should NOT last-write-win

This is another dangerous case.

PC adds:

"Great London session..."

Phone adds a completely different paragraph.

Silently picking whichever synced last could destroy someone's journal writing.

So journal body conflicts should always preserve both versions.

Example:

Journal Conflict

Main PC version

Phone version

[Keep PC]

[Keep Phone]

[Merge manually]

Atski specifically called journal bodies unsafe for last-write-wins.

9. Media files

For screenshots, I would make actual file content **immutable**.

Meaning:

Once:

media ABC

checksum = SHA256:xyz

exists, the actual binary never changes.

If the user replaces an image, create a new media asset.

Then file sync becomes easy:

Do I have checksum xyz?

Yes:

skip

No:

download it

verify checksum

Metadata can still change:

title

folder

favorite
tradeId

but the actual image blob remains stable.

That dramatically simplifies synchronization.

10. Sync cursors

We don't want:

compare 100,000 records every time

Each device remembers the last change it received from the other.

Example PC:

sync_state

peer = Blade A75
lastReceivedChangeId = 5021

Then next sync:

PC:
"Give me changes after 5021."

Phone returns:

5022
5023
5024

Fast.

Same in reverse.

What happens when both devices are online?

Ideally near-real-time:

PC saves trade

↓

local SQLite commits immediately

↓

sync_changes records update

↓

sync engine sees phone available

↓

send change

↓

phone applies change

The important part:

the PC save must never wait for the phone.

Local save happens first.

Sync happens after.

So even if connection disappears halfway through:

your data is already safe locally



What happens when both are offline?

Nothing special.

Each device keeps accumulating:

sync_changes

Phone:

change 501

change 502

change 503

PC:

change 811

change 812

When they reconnect, they exchange them.

A rule I really want

Each database transaction should do:

```
BEGIN
```

```
update domain record  
create sync_changes record
```

```
COMMIT
```

Together.

Not:

```
save trade  
...  
maybe write sync entry later
```

because the app could crash between those two operations.

SQLite transactions make this really nice.

Either:

```
both succeed
```

or:

```
neither succeeds
```

That's one reason SQLite is such a good fit.

Conflict resolution table

I'd include this in Schema v1:

```
sync_conflicts
```

id

entityType

entityId

conflictType

fieldName?

localDeviceId

remoteDeviceId

baseVersion

localPayloadJson

remotePayloadJson

detectedAt

status

UNRESOLVED

RESOLVED

resolution?

resolvedAt?

This gives us a safe inbox for messy cases.

The app doesn't have to implement the fancy conflict UI immediately.

But the data architecture is ready.

My proposed conflict policy for FXJourney v1

Data	Policy
New record created on one device	Auto-sync

Data	Policy
Different records created offline	Keep both
Immutable Analyzer reports	Keep both
Different fields edited on same record	Auto-merge
Same financial field edited	Conflict
Journal body edited both places	Conflict
Trade plan/review text edited both	Conflict
Tags/categories	Merge relationship changes
Goal metadata same-field collision	Conflict
Goal progress	Usually derived, don't sync
Habit same day edited differently	Conflict
Screenshot binary	Immutable/checksum
Screenshot metadata different fields	Merge
Delete + unchanged remote	Delete
Delete + remote edit	Conflict
Derived dashboard/analytics	Never sync

That's the core.

One more improvement: revisions

For especially important records like:

trades
journal_entries
goals

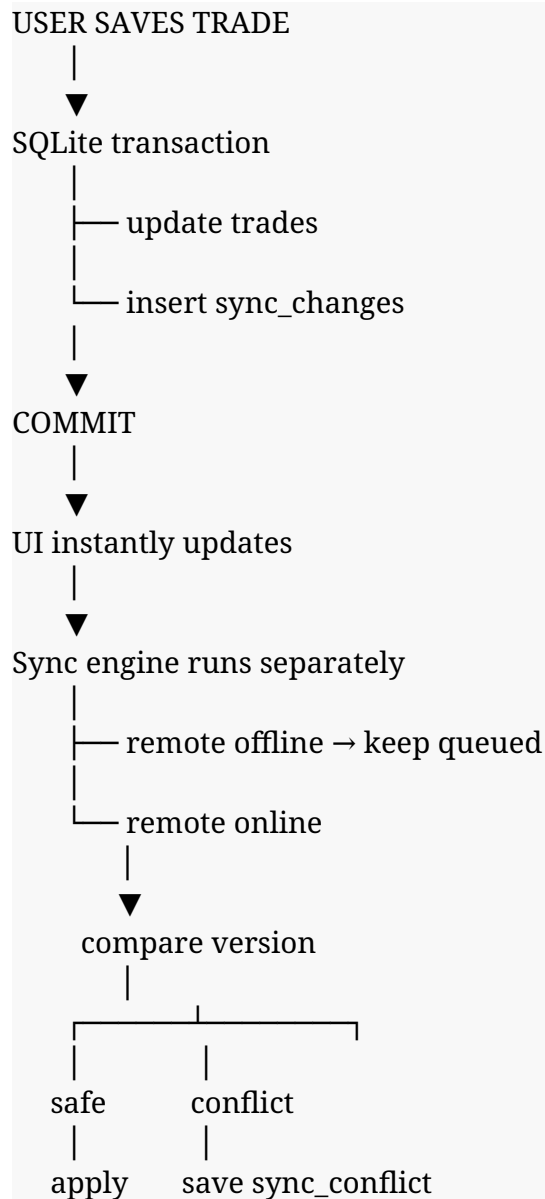
we can optionally keep a small revision history:

entity_revisions

so even after resolving a conflict, the old value isn't immediately lost.

But I would consider that **Phase 2**, because sync_conflicts already protects conflicting versions.

So the full write path becomes



THAT is how I'd solve your PC ⇌ phone requirement without making one device the boss. 🕶️🔥

And after this, I think we're close enough that the next artifact should be the **actual frozen Schema v1 specification**—all tables, fields, types, constraints, sync metadata, and conflict rules in one document before Atski implements anything.